

Mount ke google drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

Import Library

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

Load Data

```
df_3 = pd.read_csv('/content/drive/MyDrive/Praktikum/praktikum07/data/dataset_satelit.csv')
```

```
df_3.head()
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	Cu	Zn	B	b12	b11	b9	b8a	
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	5.60	16.69	26.58	0.1013	0.1922	0.4753	0.3354	0.29
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	0.65	14.47	13.60	0.1057	0.1941	0.4811	0.3208	0.28
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	2.82	14.75	29.31	0.0836	0.1750	0.4981	0.3655	0.31
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	2.53	15.32	20.24	0.0818	0.1740	0.4603	0.3671	0.32
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	1.51	15.82	27.59	0.0725	0.1598	0.4655	0.3600	0.29

```
df_3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0   No           594 non-null    int64
1   Longitude    594 non-null    float64
2   Latitude     594 non-null    float64
3   N            594 non-null    float64
4   P            594 non-null    float64
5   K            593 non-null    float64
6   Ca           594 non-null    float64
7   Mg           594 non-null    object
8   Fe           594 non-null    float64
9   Mn           594 non-null    float64
10  Cu           594 non-null    float64
11  Zn           594 non-null    float64
12  B            594 non-null    float64
13  b12          594 non-null    float64
14  b11          594 non-null    float64
15  b9           594 non-null    float64
16  b8a          594 non-null    float64
17  b8           594 non-null    float64
18  b7           594 non-null    float64
19  b6           594 non-null    float64
20  b5           594 non-null    float64
21  b4           594 non-null    float64
22  b3           594 non-null    float64
23  b2           594 non-null    float64
24  b1           594 non-null    float64
25  Sigma_VV     594 non-null    float64
26  Sigma_VH     594 non-null    float64
27  plia         594 non-null    float64
28  lia          594 non-null    float64
29  iafe         594 non-null    float64
30  gamma0_vv    594 non-null    float64
31  gamma0_vh    594 non-null    float64
32  beta0_vv     594 non-null    float64
33  beta0_vh     594 non-null    float64
dtypes: float64(32), int64(1), object(1)
```

memory usage: 157.9+ KB

df_3.describe()

	No	Longitude	Lattitude	N	P	K	Ca	Fe	Mn	Cu
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.000000	594.000000	594.000000
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.613771	308.034697	2.391195
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.579655	241.731643	1.580296
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.080000	3.160000	0.090000
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.705000	124.015000	1.172500
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.650000	239.445000	2.225000
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.372500	434.990000	3.357500
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.100000	2009.320000	8.170000

Penjelasan Kolom Statistik

1. Count = jumlah data/baris yang teredia pada setiap variabel. Nilai 594 observasi yang dianalisis.
2. Mean = Nilai rata-rata dari setiap kolom. Menggambarkan kecenderungan tengah data/ tendensi sentral data.
3. std (Standart Deviation) = Ukuran seberapa besar penyebaran data terhadap nilai rata-rata. semakin besar nilainya semakin besar variasinya.
4. Min (minimun) = nilai terkecil dalam kolom, menunjukkan batas bawah dari data.
5. 25% (Kuartil 1) = Menunjukkan bahwa 25% data memiliki nilaikurang dari atau sama dengan angka ini. Menggambarkan batar bawah distribusi data.
6. 50% (Median/Kuartil 2) = Nilai tengah dari data, separuh data berada di bawah nilai ini, dan separuh lainnya ada di atasnya.
7. 75% (Kuartil 3) = Menunjukkan bahwa 75% data berada dibawah atau sama dengan nilai ini. Menggambarkan batas atas distribusi data.
8. Max (Maximum) = Nilai terbesar dari kolom, menunjukkna batas atas dari data.

df_3.columns

```
Index(['No', 'Longitude', 'Lattitude', 'N', 'P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
      'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh'],
      dtype='object')
```

Fitur Selection

```
df_3['Mg']= pd.to_numeric(df_3['Mg'], errors='coerce')
```

```
df_3 = df_3.dropna()
```

```
# x_3= df_3[['b2', 'b3', 'b4', 'b8', 'b11', 'Sigma_VV', 'Sigma_VH']]
# y_3= df_3['N']
```

```
x_3= df_3[['P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
      'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh']]
```

```
y_3=df_3['N']
```

Split data

```
X_train, X_test, y_train, y_test = train_test_split(x_3, y_3, test_size=0.2, random_state=42)
```

modeling

```
model = LinearRegression()
#training data
```

```
model.fit(X_train, y_train)
```

LinearRegression ⓘ ?

LinearRegression()

T **B** *I* <> ↺ 📄 ☞ ☰ ☷ — Ψ 😊 ☰ Close

#machine

machine

```
#testing
y_pred = model.predict(X_test)

r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse) # Menghitung RMSE

print("R-squared Score:", r2)
print("RMSE:", rmse)
```

R-squared Score: 0.7549955124484078
RMSE: 0.19392193397227142

```
coeff = pd.DataFrame({
    'Fitur' : x_3.columns,
    'Koefisien' : model.coef_
})
print(coeff)
```

	Fitur	Koefisien
0	P	8.570837
1	K	0.056014
2	Ca	-0.058380
3	Mg	-0.161095
4	Fe	0.000284
5	Mn	0.000177
6	Cu	0.023202
7	Zn	0.002968
8	B	-0.002092
9	b12	0.327312
10	b11	-0.619437
11	b9	-0.047586
12	b8a	0.064453
13	b8	-0.072659
14	b7	0.981186
15	b6	-0.864947
16	b5	-0.329397
17	b4	1.420614
18	b3	-0.449043
19	b2	-0.357454
20	b1	-0.033418
21	Sigma_VV	0.156595
22	Sigma_VH	-1.466200
23	plia	0.021315
24	lia	-0.021906
25	iafe	-0.078122
26	gamma0_vv	0.457327
27	gamma0_vh	2.946416
28	beta0_vv	-0.569281
29	beta0_vh	-0.677599

```
import statsmodels.api as sm

X_sm = sm.add_constant(x_3) #tambahkan intercept
model_ols = sm.OLS(y_3, X_sm).fit()
print(model_ols.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	N	R-squared:	0.779			
Model:	OLS	Adj. R-squared:	0.768			
Method:	Least Squares	F-statistic:	66.10			
Date:	Mon, 03 Nov 2025	Prob (F-statistic):	6.69e-163			
Time:	08:17:21	Log-Likelihood:	156.68			
No. Observations:	592	AIC:	-251.4			
Df Residuals:	561	BIC:	-115.5			
Df Model:	30					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	3.8681	0.864	4.479	0.000	2.172	5.564

P	8.5760	0.465	18.428	0.000	7.662	9.490
K	0.0273	0.051	0.530	0.596	-0.074	0.128
Ca	-0.0416	0.023	-1.808	0.071	-0.087	0.004
Mg	-0.2166	0.063	-3.464	0.001	-0.339	-0.094
Fe	0.0002	0.000	1.102	0.271	-0.000	0.001
Mn	0.0002	4.59e-05	4.083	0.000	9.72e-05	0.000
Cu	0.0229	0.006	4.046	0.000	0.012	0.034
Zn	0.0024	0.001	1.889	0.059	-9.52e-05	0.005
B	-0.0022	0.001	-1.476	0.141	-0.005	0.001
b12	0.2010	0.534	0.377	0.707	-0.848	1.250
b11	-0.9945	0.517	-1.922	0.055	-2.011	0.022
b9	-0.0802	0.067	-1.191	0.234	-0.212	0.052
b8a	0.0962	0.067	1.427	0.154	-0.036	0.229
b8	-0.0806	0.024	-3.362	0.001	-0.128	-0.034
b7	1.3059	0.542	2.409	0.016	0.241	2.371
b6	-0.9238	0.462	-2.001	0.046	-1.831	-0.017
b5	-0.5470	0.488	-1.120	0.263	-1.506	0.412
b4	1.2561	0.972	1.292	0.197	-0.653	3.166
b3	0.3085	1.639	0.188	0.851	-2.910	3.527
b2	-0.7075	0.942	-0.751	0.453	-2.559	1.144
b1	0.0023	0.352	0.006	0.995	-0.689	0.694
Sigma_VV	0.9543	0.889	1.074	0.283	-0.792	2.700
Sigma_VH	-1.1666	0.861	-1.356	0.176	-2.857	0.524
plia	-0.0113	0.071	-0.159	0.873	-0.150	0.128
lia	0.0102	0.071	0.144	0.886	-0.129	0.149
iafe	-0.0802	0.023	-3.475	0.001	-0.126	-0.035
gamma0_vv	-0.5040	2.180	-0.231	0.817	-4.785	3.777
gamma0_vh	3.1827	4.643	0.685	0.493	-5.938	12.303
beta0_vv	-0.2381	1.580	-0.151	0.880	-3.342	2.865
beta0_vh	-0.5472	3.536	-0.155	0.877	-7.493	6.399

```

=====
Omnibus:                28.388    Durbin-Watson:           1.527
Prob(Omnibus):          0.000    Jarque-Bera (JB):        53.216
Skew:                   -0.310    Prob(JB):                2.78e-12
Kurtosis:               4.332    Cond. No.                2.98e+05
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.98e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.